



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА –Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Дисциплина «Технологии извлечения знаний из больших данных»

Отчет

о проделанной практической работе №8

Выполнил студент
Группы: ББМО-01-25
Железняков Родион
Алексеевич

Москва
2025

Теория

Данная работа посвящена практическому освоению нейронных сетей прямого распространения (feedforward neural networks) на примере нескольких классических задач машинного обучения. Используется библиотека Scikit-learn, в частности модуль MLPClassifier (многослойный перцептрон) и MLPRegressor для задач регрессии.

1. Исключающее ИЛИ (XOR)

XOR — нелинейно разделимая функция, которую невозможно решить однослойным перцептроном (теорема Минского–Пейперта). Для её решения требуется сеть с хотя бы одним скрытым слоем.

Исследуется влияние архитектуры (количество слоёв и нейронов), скорости обучения (learning_rate) и числа эпох на ошибку (MSE). Показано, что даже небольшая сеть (например, с 3 нейронами в скрытом слое) может идеально решить задачу.

2. Определение направления двоичного сдвига

Задача классификации последовательностей, где сеть должна научиться распознавать направление циклического сдвига битовой строки.

Используется полносвязная сеть с одним скрытым слоем, исследуется влияние количества нейронов и числа эпох на точность. Задача оказалась сложной для простой архитектуры, что демонстрирует ограниченность модели при малом объёме данных.

3. Распознавание символов (X, Y, I, C)

Задача классификации образов, представленных в виде бинарных матриц 3×3 . Нейросеть выступает в роли паттерн-распознавателя.

Исследуется устойчивость сети к бинарному шуму (инверсии пикселей). Показано, что с увеличением уровня шума точность закономерно падает, что характерно для моделей, обученных на идеальных данных.

4. Искусственный нос (химический анализ)

Многоклассовая классификация на основе данных от химических сенсоров. Требуется нормализация данных и кодирование меток (One-Hot Encoding).

Оптимизируется количество нейронов в скрытом слое для минимизации MSE. Используется функция активации tanh и SGD-оптимизатор.

5. Прогнозирование временных рядов

Нейросеть применяется для регрессионного прогнозирования на основе скользящего окна. Используется архитектура MLPRegressor.

Сравниваются:

Однофакторный прогноз (на основе одного ряда)

Многофакторный прогноз (с несколькими признаками)

Сравнение с линейной моделью (МГУА) — показано, что для малых данных линейные методы могут быть эффективнее.

ИСКЛЮЧАЮЩЕЕ ИЛИ.

1. Создание и обучение простейшей нейронной сети Цель – освоение основных приемов работы с НС в ходе создания и обучения простейшей нейронной сети. Задание
2. Создать и обучить нейронную сеть, которая будет способна решать логическую задачу исключающего «ИЛИ». Таблица истинности для весьма полезной логической функции приведена в табл. 1.
3. Проверить работоспособность нейронной сети.
4. Исследовать различные варианты настройки НС и ошибку обучения.
5. Исследовать различные архитектуры НС: с одним скрытым слоем и разным количеством нейронов и с 2 скрытыми слоями и разным количеством нейронов в каждом скрытом слое

```
1 # Импорты
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import pandas as pd
6 from sklearn.neural_network import MLPClassifier
7 from sklearn.metrics import accuracy_score, mean_squared_error
8 from itertools import product
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 # Настройка стиля графиков
13 sns.set(style="whitegrid")
14 plt.rcParams['figure.figsize'] = (10, 6)
```

```
1 X = np.array([[0, 0],
2               [0, 1],
3               [1, 0],
4               [1, 1]])
5 y = np.array([0, 1, 1, 0]) # XOR
```

```

1 print("=== Проверка базовой работоспособности ===")
2 mlp = MLPClassifier(
3     hidden_layer_sizes=(4,),      # один скрытый слой с 4 нейрона
4     activation='tanh',
5     solver='sgd',
6     learning_rate_init=0.1,
7     max_iter=10000,
8     random_state=42,
9     verbose=False
10 )
11
12 mlp.fit(X, y)
13 y_pred = mlp.predict(X)
14 print("Предсказания:", y_pred)
15 print("Точность:", accuracy_score(y, y_pred))
16 print("MSE:", mean_squared_error(y, y_pred))

```

```

=== Проверка базовой работоспособности ===
Предсказания: [0 1 1 0]
Точность: 1.0
MSE: 0.0

```

```

1 print("\n=== Исследование: learning_rate vs max_iter ===")
2
3 learning_rates = np.logspace(-4, 0, 5) # от 0.0001 до 1
4 max_iters = np.linspace(100, 5000, 5, dtype=int)
5
6 errors = np.zeros((len(learning_rates), len(max_iters)))
7
8 for i, lr in enumerate(learning_rates):
9     for j, epochs in enumerate(max_iters):
10         mlp = MLPClassifier(
11             hidden_layer_sizes=(4,),
12             activation='tanh',
13             solver='sgd',
14             learning_rate_init=lr,
15             max_iter=epochs,
16             random_state=42,
17             tol=1e-10, # отключаем раннюю остановку
18             n_iter_no_change=10000 # чтобы обучалось до max_it
19         )
20         mlp.fit(X, y)
21         y_pred = mlp.predict(X)
22         errors[i, j] = mean_squared_error(y, y_pred)

```

```

=== Исследование: learning_rate vs max_iter ===

```

```

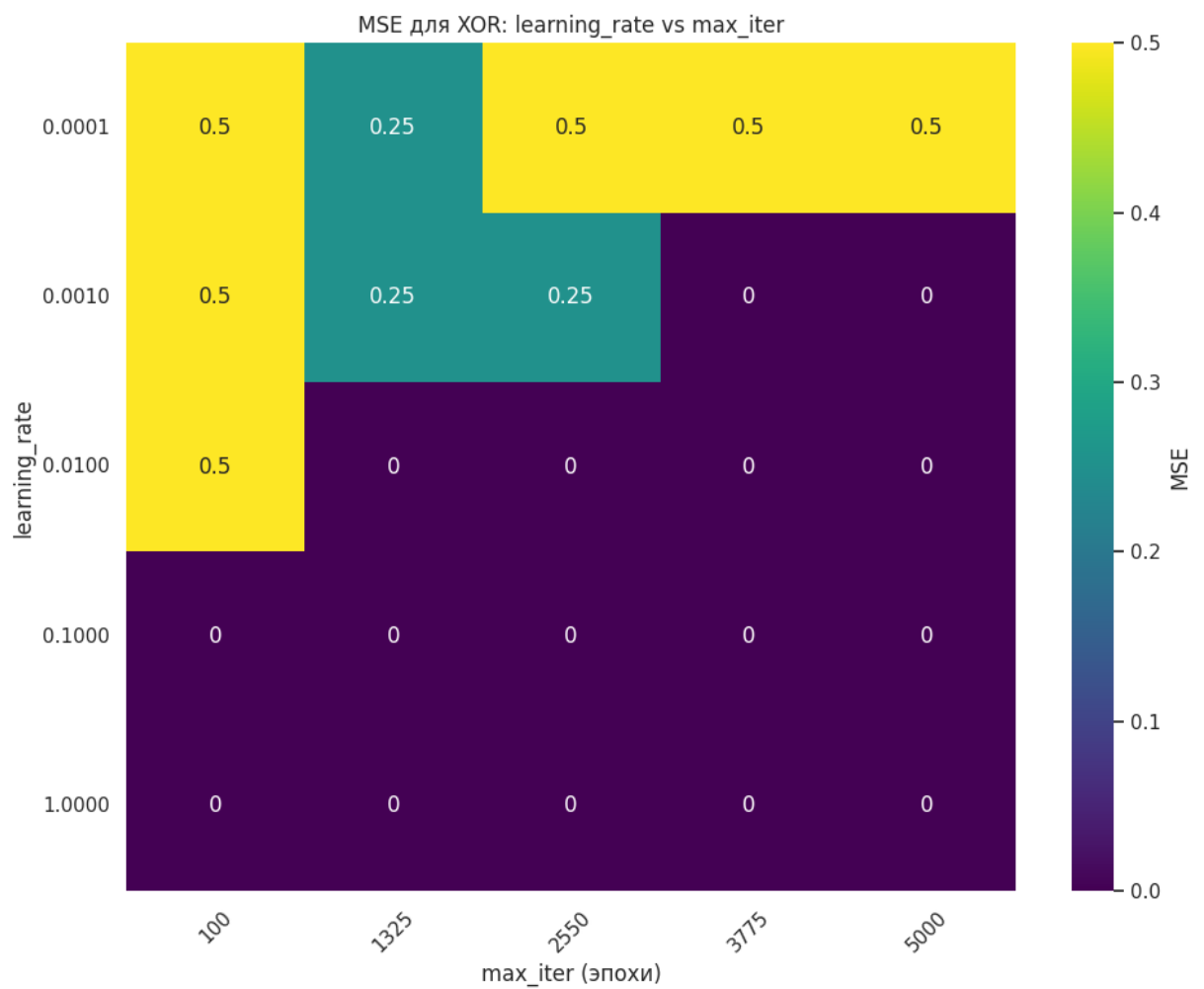
1 # Построение heatmap
2 plt.figure(figsize=(10, 8))

```

```

3 sns.heatmap(
4     errors,
5     xticklabels=[f"{it}" for it in max_iters],
6     yticklabels=[f"{lr:.4f}" for lr in learning_rates],
7     cmap="viridis",
8     cbar_kws={'label': 'MSE'},
9     annot=True
10 )
11 plt.title("MSE для XOR: learning_rate vs max_iter")
12 plt.xlabel("max_iter (эпохи)")
13 plt.ylabel("learning_rate")
14 plt.xticks(rotation=45)
15 plt.yticks(rotation=0)
16 plt.tight_layout()
17 plt.show()

```



```

1 print("\n=== Исследование архитектур НС ===")
2
3 # Варианты архитектур:
4 # - Один скрытый слой: от 1 до 10 нейронов
5 # - Два скрытых слоя: от 1 до 6 нейронов в каждом
6
7 # Сначала исследуем один слой
8 neurons_1 = np.arange(1, 11)
9 errors_1layer = []
10
11 for n in neurons_1:
12     mlp = MLPClassifier(
13         hidden_layer_sizes=(n,),
14         activation='tanh',
15         solver='sgd',
16         learning_rate_init=0.1,
17         max_iter=100,
18         random_state=42,
19         tol=1e-10,
20         n_iter_no_change=5000
21     )
22     mlp.fit(X, y)
23     y_pred = mlp.predict(X)
24     errors_1layer.append(mean_squared_error(y, y_pred))
25
26 # Теперь два скрытых слоя
27 neurons_range = np.arange(1, 7)
28 errors_2layer = np.zeros((len(neurons_range), len(neurons_range)))
29
30 for i, n1 in enumerate(neurons_range):
31     for j, n2 in enumerate(neurons_range):
32         mlp = MLPClassifier(
33             hidden_layer_sizes=(n1, n2),
34             activation='tanh',
35             solver='sgd',
36             learning_rate_init=0.1,
37             max_iter=100,
38             random_state=42,
39             tol=1e-10,
40             n_iter_no_change=5000
41         )
42         mlp.fit(X, y)
43         y_pred = mlp.predict(X)
44         errors_2layer[i, j] = mean_squared_error(y, y_pred)

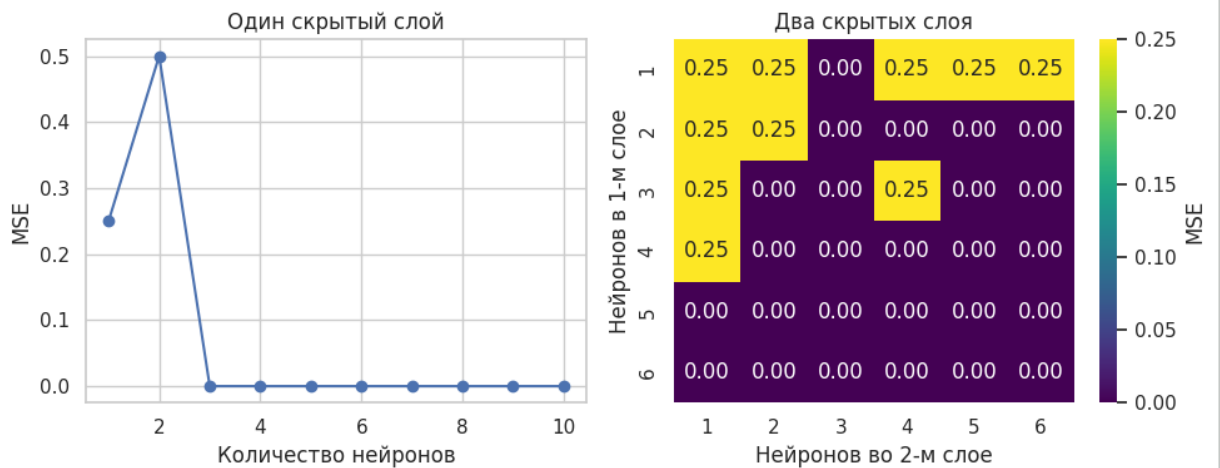
```

=== Исследование архитектур НС ===

```

1 # Построим график для одного слоя
2 plt.figure(figsize=(10, 4))
3 plt.subplot(1, 2, 1)
4 plt.plot(neurons_1, errors_1layer, marker='o')
5 plt.title("Один скрытый слой")
6 plt.xlabel("Количество нейронов")
7 plt.ylabel("MSE")
8 plt.grid(True)
9
10
11 # Heatmap для двух слоёв
12 plt.subplot(1, 2, 2)
13 sns.heatmap(
14     errors_2layer,
15     xticklabels=neurons_range,
16     yticklabels=neurons_range,
17     cmap="viridis",
18     annot=True,
19     fmt=".2f",
20     cbar_kws={'label': 'MSE'}
21 )
22 plt.title("Два скрытых слоя")
23 plt.xlabel("Нейронов во 2-м слое")
24 plt.ylabel("Нейронов в 1-м слое")
25 plt.tight_layout()
26 plt.show()

```



```

1 min_error_1 = min(errors_1layer)
2 best_n_1 = neurons_1[np.argmin(errors_1layer)]
3
4 min_error_2 = errors_2layer.min()
5 best_idx_2 = np.unravel_index(np.argmin(errors_2layer), errors_
6 best_arch_2 = (neurons_range[best_idx_2[0]], neurons_range[best
7
8 print(f"\nЛучшая архитектура с 1 слоем: {best_n_1} нейронов, MS
9 print(f"Лучшая архитектура с 2 слоями: {best_arch_2} нейронов,

```

Лучшая архитектура с 1 слоем: 3 нейронов, MSE = 0.000000
 Лучшая архитектура с 2 слоями: (np.int64(1), np.int64(3)) нейронов,

ОПРЕДЕЛЕНИЕ НАПРАВЛЕНИЯ ДВОИЧНОГО СДВИГА

Цель – построение, обучение и тестирование нейронной сети, предназначенной для определения направления сдвига двоичного кода.

Задание

1. Создать и обучить нейронную сеть для определения направление циклического сдвига четырехпозиционного двоичного кода.
2. Оптимизация структуры нейронной сети по критерию минимума ошибки обучения (на основе количества нейронов в скрытом слое и по количеству эпох(итераций)).
3. Проверить работоспособность нейронной сети.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.neural_network import MLPClassifier
4 from sklearn.metrics import accuracy_score, mean_squared_error
5 from itertools import product

```



```

1 # Все 4-битные последовательности
2 all_sequences = [''.join(p) for p in product('01', repeat=4)]
3 forbidden = {'0000', '1111', '1010', '0101'}
4 allowed = [s for s in all_sequences if s not in forbidden]
5
6 # Функции циклического сдвига
7 def cyclic_left_shift(s):
8     return s[1:] + s[0]
9
10 def cyclic_right_shift(s):
11     return s[-1] + s[:-1]
12
13 # Выбираем 6 обучающих последовательностей
14 train_sequences = allowed[:6]
15 test_sequences = allowed[6:] # остальные – для теста
16
17 # Создание обучающего набора
18 X_train, y_train = [], []
19 for seq in train_sequences:
20     # Левый сдвиг → метка 0
21     X_train.append([int(b) for b in seq + cyclic_left_shift(seq)
22     y_train.append(0)
23     # Правый сдвиг → метка 1
24     X_train.append([int(b) for b in seq + cyclic_right_shift(seq)
25     y_train.append(1)
26
27 X_train = np.array(X_train)
28 y_train = np.array(y_train)
29
30 # Создание тестового набора
31 X_test, y_test = [], []
32 for seq in test_sequences:
33     X_test.append([int(b) for b in seq + cyclic_left_shift(seq)
34     y_test.append(0)
35     X_test.append([int(b) for b in seq + cyclic_right_shift(seq)
36     y_test.append(1)
37
38 X_test = np.array(X_test)
39 y_test = np.array(y_test)
40
41 print("Размер обучающей выборки:", X_train.shape)
42 print("Размер тестовой выборки:", X_test.shape)

```

Размер обучающей выборки: (12, 8)

Размер тестовой выборки: (12, 8)

```

1 hidden_neurons_range = list(range(1, 10)) # от 1 до 10 нейроно
2 epochs_range = np.linspace(1000, 5000, 5, dtype=int)
3
4 # Матрицы для хранения ошибок и точности
5 errors = np.zeros((len(hidden_neurons_range), len(epochs_range))
6 accuracies = np.zeros((len(hidden_neurons_range), len(epochs_ra
7
8 # Перебор гиперпараметров
9 for i, n_neurons in enumerate(hidden_neurons_range):
10     for j, n_epochs in enumerate(epochs_range):
11         mlp = MLPClassifier(
12             hidden_layer_sizes=(n_neurons, ),
13             activation='tanh',
14             solver='sgd',
15             learning_rate_init=0.1,
16             max_iter=n_epochs,
17             random_state=42,
18             tol=1e-10,
19             n_iter_no_change=10000
20         )
21         mlp.fit(X_train, y_train)
22         y_pred = mlp.predict(X_test)
23         errors[i, j] = mean_squared_error(y_test, y_pred)
24         accuracies[i, j] = accuracy_score(y_test, y_pred)

```

```

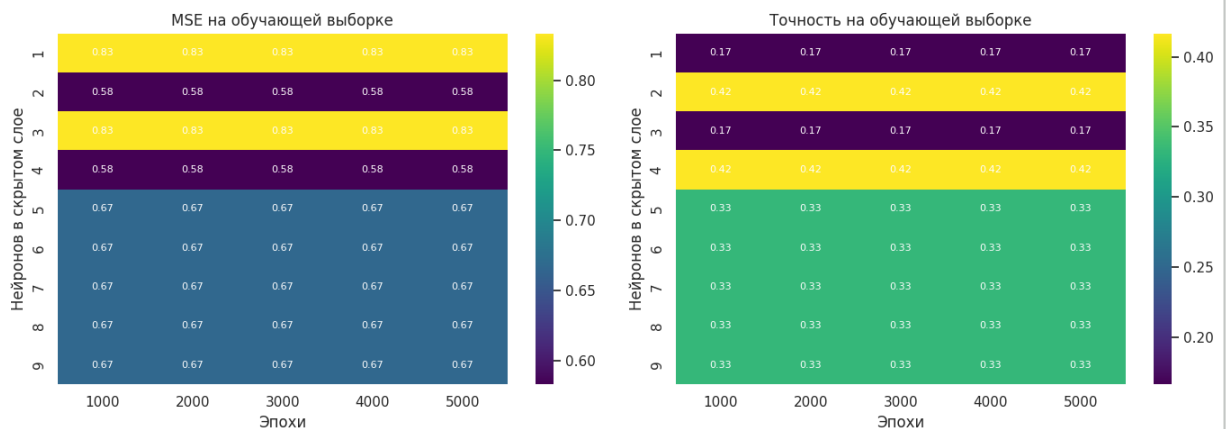
1 # Преобразуем массивы в DataFrame для удобства использования с
2 errors_df = pd.DataFrame(
3     errors,
4     index=hidden_neurons_range,
5     columns=epochs_range
6 )
7
8 accuracies_df = pd.DataFrame(
9     accuracies,
10    index=hidden_neurons_range,
11    columns=epochs_range
12 )
13
14 # Создаём фигуру с двумя подграфиками
15 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
16
17 # Heatmap для MSE
18 sns.heatmap(
19     errors_df,
20     annot=True,
21     fmt=".2f",
22     cmap="viridis",
23     cbar=True,
24     ax=axes[0],
25     annot_kws={"color": "white", "fontsize": 8}

```

```

26 )
27 axes[0].set_title('MSE на обучающей выборке')
28 axes[0].set_xlabel('Эпохи')
29 axes[0].set_ylabel('Нейронов в скрытом слое')
30
31 # Heatmap для точности
32 sns.heatmap(
33     accuracies_df,
34     annot=True,
35     fmt=".2f",
36     cmap="viridis",
37     cbar=True,
38     ax=axes[1],
39     annot_kws={"color": "white", "fontsize": 8}
40 )
41 axes[1].set_title('Точность на обучающей выборке')
42 axes[1].set_xlabel('Эпохи')
43 axes[1].set_ylabel('Нейронов в скрытом слое')
44
45 plt.tight_layout()
46 plt.show()

```



```

1 # Находим лучшую комбинацию по MSE
2 best_i, best_j = np.unravel_index(np.argmin(errors), errors.sha

```

```

3 best_neurons = hidden_neurons_range[best_i]
4 best_epochs = epochs_range[best_j]
5
6 print(f"\nЛучшая архитектура:")
7 print(f"  Нейронов: {best_neurons}")
8 print(f"  Эпох: {best_epochs}")
9 print(f"  MSE (train): {errors[best_i, best_j]:.6f}")
10 print(f"  Точность (train): {accuracies[best_i, best_j]:.6f}")
11
12 # Обучаем финальную модель
13 final_model = MLPClassifier(
14     hidden_layer_sizes=(best_neurons,),
15     activation='tanh',
16     solver='sgd',
17     learning_rate_init=0.1,
18     max_iter=best_epochs,
19     random_state=42,
20     tol=1e-10,
21     n_iter_no_change=10000
22 )
23 final_model.fit(X_train, y_train)
24
25 # Проверка на тестовой выборке
26 y_test_pred = final_model.predict(X_test)
27 test_accuracy = accuracy_score(y_test, y_test_pred)
28 test_mse = mean_squared_error(y_test, y_test_pred)
29
30 print(f"\nРезультаты на тестовой выборке:")
31 print(f"  Точность: {test_accuracy:.6f}")
32 print(f"  MSE: {test_mse:.6f}")
33
34 # Демонстрация предсказаний
35 print("\nПримеры предсказаний на тестовых данных:")
36 for i in range(12):
37     orig = ''.join(map(str, X_test[i][:4]))
38     shifted = ''.join(map(str, X_test[i][4:]))
39     true_dir = "<-" if y_test[i] == 0 else "->"
40     pred_dir = "<-" if y_test_pred[i] == 0 else "->"
41     status = "+" if y_test[i] == y_test_pred[i] else "-"
42     print(f"{orig} → {shifted} | Истинное: {true_dir}, Предсказ")

```

Лучшая архитектура:

Нейронов: 2

Эпох: 1000

MSE (train): 0.583333

Точность (train): 0.416667

Результаты на тестовой выборке:

Точность: 0.416667

MSE: 0.583333

Примеры предсказаний на тестовых данных:

1000 → 0001	Истинное: <-, Предсказано: -> -
1000 → 0100	Истинное: ->, Предсказано: <- -
1001 → 0011	Истинное: <-, Предсказано: -> -
1001 → 1100	Истинное: ->, Предсказано: <- -
1011 → 0111	Истинное: <-, Предсказано: <- +
1011 → 1101	Истинное: ->, Предсказано: -> +
1100 → 1001	Истинное: <-, Предсказано: -> -
1100 → 0110	Истинное: ->, Предсказано: -> +
1101 → 1011	Истинное: <-, Предсказано: -> -
1101 → 1110	Истинное: ->, Предсказано: <- -
1110 → 1101	Истинное: <-, Предсказано: <- +
1110 → 0111	Истинное: ->, Предсказано: -> +

РАСПОЗНАВАНИЕ СИМВОЛОВ

Цель – разработать и исследовать нейронную сеть обратного распространения, предназначенную для распознавания образов. Задание

1. Построить и обучить нейронную сеть, которая могла бы решать задачу распознавания символов: X, Y, I, C.
2. Найти оптимальную структуру НС, минимизировав ошибку обучения в зависимости от количества нейронов в скрытом слое
3. Произвести тестирование нейронной сети при добавлении шума.
4. Построить нейронную сеть в соответствии с заданиями и выполнить задания 2-3.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.neural_network import MLPClassifier
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, classification_repo
6 import itertools
```

```

1 # Определяем символы в виде матриц 3x3 (1 – пиксель активен, 0
2 symbols = {
3     'X': np.array([[1, 0, 1],
4                     [0, 1, 0],
5                     [1, 0, 1]]),
6     'Y': np.array([[1, 0, 1],
7                     [0, 1, 0],
8                     [0, 1, 0]]),
9     'I': np.array([[0, 1, 0],
10                    [0, 1, 0],
11                    [0, 1, 0]]),
12     'C': np.array([[1, 1, 1],
13                     [1, 0, 0],
14                     [1, 1, 1]])
15 }
16
17 # Преобразуем в векторы длиной 9 (растровое представление)
18 X = []
19 y = []
20
21 for label, matrix in symbols.items():
22     X.append(matrix.flatten())
23     y.append(label)
24
25 X = np.array(X)
26 y = np.array(y)
27
28 print("Массив признаков X (размерность 4x9):")
29 print(X)
30 print("\nМетки y:")
31 print(y)

```

Массив признаков X (размерность 4x9):

```

[[1 0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 0 1 0]
 [0 1 0 0 1 0 0 1 0]
 [1 1 1 1 0 0 1 1 1]]

```

Метки y:

```
['X' 'Y' 'I' 'C']
```

```

1 # Визуализация исходных символов
2 fig, axes = plt.subplots(1, 4, figsize=(8, 2))
3 for ax, (label, mat) in zip(axes, symbols.items()):
4     ax.imshow(mat, cmap='gray_r', vmin=0, vmax=1)
5     ax.set_title(label)
6     ax.axis('off')
7 plt.suptitle('Исходные символы')
8 plt.show()

```



```

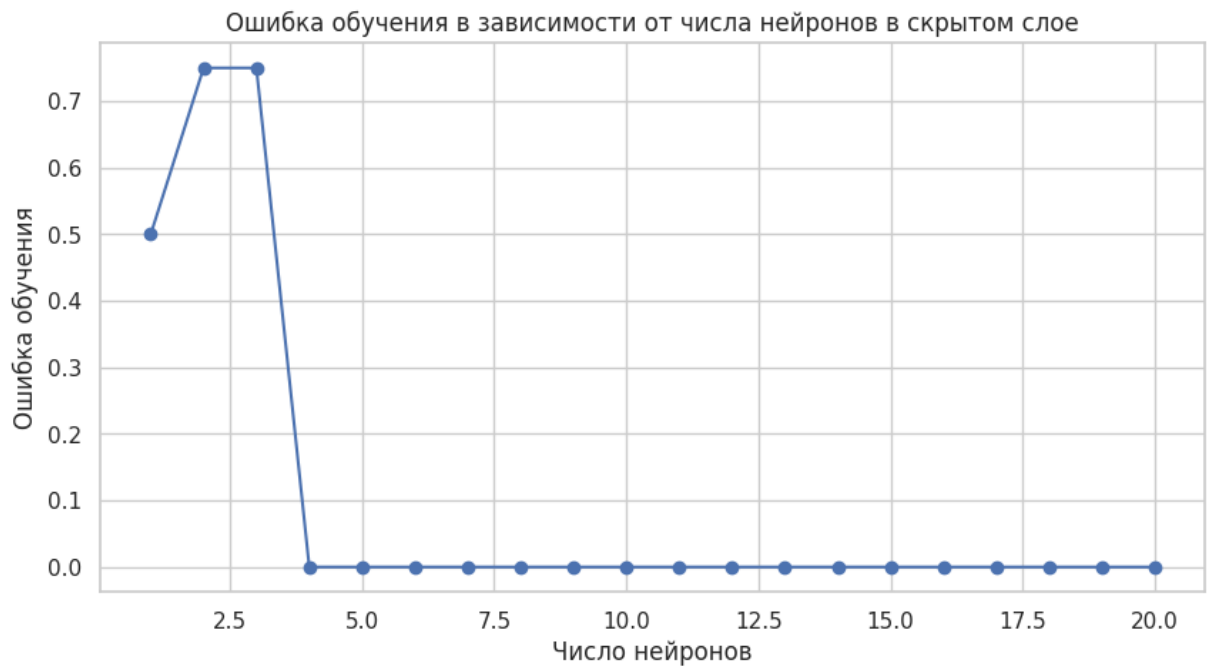
1 # Проверим разные количества нейронов в скрытом слое
2 hidden_layer_sizes = range(1, 21) # от 1 до 20 нейронов
3 train_errors = []
4 test_errors = []
5
6 for n_neurons in hidden_layer_sizes:
7     mlp = MLPClassifier(hidden_layer_sizes=(n_neurons,),
8                         max_iter=1000,
9                         random_state=42,
10                        solver='adam',
11                        learning_rate_init=0.01)
12
13     # Обучаем модель
14     mlp.fit(X, y)
15
16     # Предсказание на обучающей выборке
17     y_pred = mlp.predict(X)
18
19     # Ошибки
20     train_error = 1 - accuracy_score(y, y_pred)
21     train_errors.append(train_error)

```

```

1 # Визуализация ошибки в зависимости от числа нейронов
2 plt.figure(figsize=(10, 5))
3 plt.plot(hidden_layer_sizes, train_errors, marker='o')
4 plt.title('Ошибка обучения в зависимости от числа нейронов в ск
5 plt.xlabel('Число нейронов')
6 plt.ylabel('Ошибка обучения')
7 plt.grid(True)
8 plt.show()
9
10 # Найдём наилучшее число нейронов (минимальная ошибка)
11 best_n = hidden_layer_sizes[np.argmin(train_errors)]
12 print(f"Оптимальное число нейронов: {best_n} (ошибка: {min(trai

```



Оптимальное число нейронов: 4 (ошибка: 0.0000)


```

1 # Обучаем финальную модель
2 mlp_final = MLPClassifier(hidden_layer_sizes=(best_n,),
3                             max_iter=1000,
4                             random_state=42,
5                             solver='adam',
6                             learning_rate_init=0.01)
7
8 mlp_final.fit(X, y)
9
10 # Проверка на обучающих данных
11 y_pred_final = mlp_final.predict(X)
12 print("Результаты на обучающей выборке:")
13 print(classification_report(y, y_pred_final))

```

Результаты на обучающей выборке:

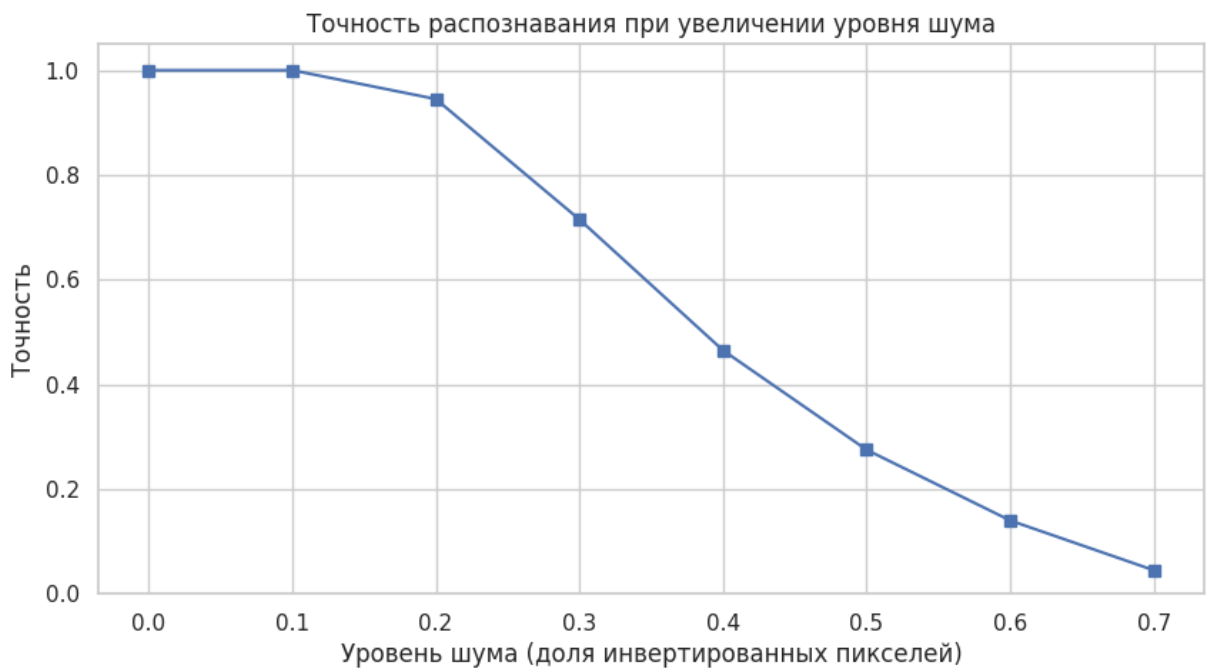
	precision	recall	f1-score	support
C	1.00	1.00	1.00	1
I	1.00	1.00	1.00	1
X	1.00	1.00	1.00	1
Y	1.00	1.00	1.00	1
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

```
1 def add_noise(image, noise_level=0.1):
2     """
3     Добавляет случайный бинарный шум к изображению.
4     noise_level – доля инвертированных пикселей (0..1)
5     """
6     image_noisy = image.copy()
7     n_pixels = image.size
8     n_noisy = int(noise_level * n_pixels)
9     indices = np.random.choice(n_pixels, n_noisy, replace=False)
10    image_noisy[indices] = 1 - image_noisy[indices] # инвертир
11    return image_noisy
12
13 # Генерация зашумленных образцов
14 noise_levels = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]
15 n_tests = 100 # количество тестов на каждый уровень шума
16
17 results = {}
18
19 for noise in noise_levels:
20     correct = 0
21     for _ in range(n_tests):
22         for label, matrix in symbols.items():
23             clean = matrix.flatten()
24             noisy = add_noise(clean, noise_level=noise)
25             pred = mlp_final.predict([noisy])[0]
26             if pred == label:
27                 correct += 1
28     accuracy = correct / (n_tests * len(symbols))
29     results[noise] = accuracy
```

```

1 # Визуализация устойчивости к шуму
2 plt.figure(figsize=(10, 5))
3 plt.plot(list(results.keys()), list(results.values()), marker='
4 plt.title('Точность распознавания при увеличении уровня шума')
5 plt.xlabel('Уровень шума (доля инвертированных пикселей)')
6 plt.ylabel('Точность')
7 plt.grid(True)
8 plt.ylim(0, 1.05)
9 plt.show()
10
11 print("\nТочность при различных уровнях шума:")
12 for noise, acc in results.items():
13     print(f"Шум {noise:.1f}: {acc:.4f}")

```



Точность при различных уровнях шума:

```

Шум 0.0: 1.0000
Шум 0.1: 1.0000
Шум 0.2: 0.9450
Шум 0.3: 0.7150
Шум 0.4: 0.4650
Шум 0.5: 0.2750
Шум 0.6: 0.1400
Шум 0.7: 0.0450

```

ИСКУССТВЕННЫЙ НОС

Цель – разработать и исследовать ИНС обратного распространения для искусственного носа, предназначенного для химического анализа воздушной среды . Задание

1. Исследовать и проанализировать имеющиеся экспериментальные данные (табл. 4.1), и определить количество вводов и выводов, требуемых для полносвязанной ИНС обратного распространения.
2. Создать и обучить нейронную сеть, которая будет способна указывать наличие определенных примесей в воздухе при анализе показаний химических датчиков.
3. Обучить нейронную сеть, расшив количество представительских выборок (обучающих пар), применяемых для обучения ИНС (табл. 4.2).
4. Определить оптимальную структуру нейронной сети с точки зрения минимизации среднеквадратической ошибки обучения.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.neural_network import MLPClassifier
4 from sklearn.metrics import mean_squared_error
5 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
6 from sklearn.model_selection import train_test_split
```

```
1
2 X_raw = np.array([
3     [1.0, 0.05, 0.1, 0.3, 0.07, 0.08, 0.2, 0.05, 0.2, 0.6, 0.8]
4     [1.0, 0.05, 0.1, 0.3, 0.07, 0.08, 0.2, 0.05, 0.2, 0.6, 0.8]
5     [0.8, 0.4, 0.7, 0.6, 0.1, 0.5, 1.0, 0.75, 0.5, 0.7, 0.8],
6     [0.9, 0.2, 0.4, 0.5, 0.1, 0.7, 0.6, 0.5, 0.5, 0.7, 0.8],
7     [0.85, 0.7, 0.8, 0.65, 0.1, 0.4, 1.0, 0.7, 0.4, 0.6, 0.7],
8     [0.9, 0.3, 0.3, 0.4, 0.04, 0.1, 0.5, 0.3, 0.2, 0.7, 0.8],
9     [0.95, 0.18, 0.21, 0.3, 0.05, 0.1, 0.3, 0.2, 0.2, 0.5, 0.7]
10 ])
11
12
13 y_labels = np.array([
14     "Уксус",
15     "Нет",
16     "Ацетон",
17     "Аммиак",
18     "Изопропанол",
19     "Белый штрих",
20     "Уксус"
21 ])
22
23 encoder = OneHotEditor = OneHotEncoder(sparse_output=False)
24 y_onehot = encoder.fit_transform(y_labels.reshape(-1, 1)) # sh
25
26
27 X_train, X_test, y_train, y_test = train_test_split(
28     X_raw, y_onehot, test_size=0.2, random_state=42
29 )
```

```

1 hidden_neurons_range = range(1, 21)
2 mse_list = []
3 models = {}
4
5 print("Обучение моделей с разным числом нейронов в скрытом слое")
6 for n_neurons in hidden_neurons_range:
7     mlp = MLPClassifier(
8         hidden_layer_sizes=(n_neurons,),
9         activation='tanh',
10        solver='sgd',
11        learning_rate_init=0.01,
12        max_iter=1000,
13        random_state=42,
14        tol=1e-6,
15        n_iter_no_change=100,
16        early_stopping=False
17    )
18
19    mlp.fit(X_raw, y_labels)
20
21    y_pred_proba = mlp.predict_proba(X_raw)
22
23    mse = mean_squared_error(y_onehot, y_pred_proba)
24    mse_list.append(mse)
25    models[n_neurons] = mlp
26
27    print(f"Нейронов: {n_neurons:2d} | MSE: {mse:.6f}")
28

```

Обучение моделей с разным числом нейронов в скрытом слое...

```

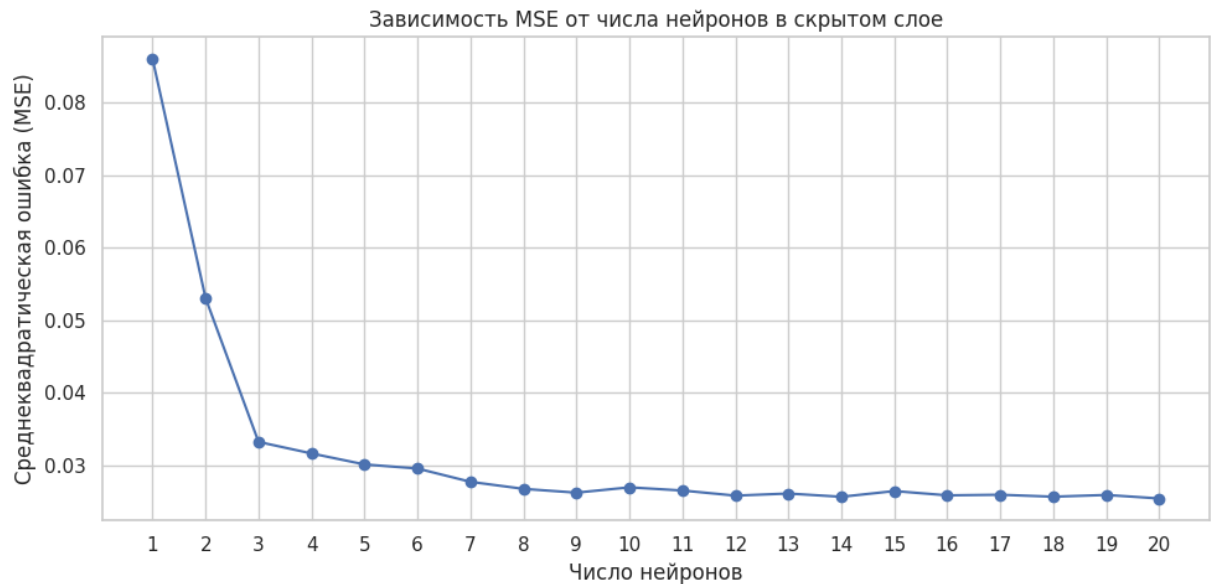
Нейронов: 1 | MSE: 0.086011
Нейронов: 2 | MSE: 0.053064
Нейронов: 3 | MSE: 0.033230
Нейронов: 4 | MSE: 0.031638
Нейронов: 5 | MSE: 0.030125
Нейронов: 6 | MSE: 0.029559
Нейронов: 7 | MSE: 0.027727
Нейронов: 8 | MSE: 0.026756
Нейронов: 9 | MSE: 0.026232
Нейронов: 10 | MSE: 0.026969
Нейронов: 11 | MSE: 0.026528
Нейронов: 12 | MSE: 0.025825
Нейронов: 13 | MSE: 0.026113
Нейронов: 14 | MSE: 0.025665
Нейронов: 15 | MSE: 0.026454
Нейронов: 16 | MSE: 0.025864
Нейронов: 17 | MSE: 0.025946
Нейронов: 18 | MSE: 0.025675
Нейронов: 19 | MSE: 0.025916
Нейронов: 20 | MSE: 0.025431

```

```

1 plt.figure(figsize=(10, 5))
2 plt.plot(hidden_neurons_range, mse_list, marker='o')
3 plt.title('Зависимость MSE от числа нейронов в скрытом слое')
4 plt.xlabel('Число нейронов')
5 plt.ylabel('Среднеквадратическая ошибка (MSE)')
6 plt.grid(True)
7 plt.xticks(hidden_neurons_range)
8 plt.tight_layout()
9 plt.show()

```



```

1 optimal_neurons = hidden_neurons_range[np.argmin(mse_list)]
2 best_model = models[optimal_neurons]
3 print(f"Оптимальное число нейронов: {optimal_neurons}")
4 print(f"Минимальная MSE: {min(mse_list):.6f}")

```

Оптимальное число нейронов: 20
Минимальная MSE: 0.025431

```

1 print("Проверка предсказаний на обучающих данных:")
2 for i, x in enumerate(X_raw):
3     pred = best_model.predict([x])[0]
4     proba = best_model.predict_proba([x])[0]
5     true_label = y_labels[i]
6     print(f"Образец {i+1}: Истинно – {true_label:12s} | Предска

```

Проверка предсказаний на обучающих данных:

Образец 1: Истинно – Уксус	Предсказано – Уксус	Уве
Образец 2: Истинно – Нет	Предсказано – Уксус	Уве
Образец 3: Истинно – Ацетон	Предсказано – Ацетон	Уве
Образец 4: Истинно – Аммиак	Предсказано – Аммиак	Уве
Образец 5: Истинно – Изопропанол	Предсказано – Изопропанол	Уве
Образец 6: Истинно – Белый штрих	Предсказано – Белый штрих	Уве
Образец 7: Истинно – Уксус	Предсказано – Уксус	Уве

ПРОГНОЗИРОВАНИЕ

Цель – разработать и исследовать нейронную сеть обратного распространения, предназначенную для прогнозирования временных серий, а также для анализа качества генератора случайных чисел. Задание

1. Создать и обучить нейронную сеть, предназначенную для анализа временных серий заданной размерности и отражающую структуру данных серий. Окно скользящего равна 25, глубина прогноза: 2, число обучающих выборок: 8.
2. Осуществить прогноз значений будущих элементов временных серий.
3. Проверить работу НС и определить точность прогноза по формуле:
4. Создать и обучить две нейронные сети: предназначенную для анализа временных серий заданной размерности (окно скользящего равна 4, шаг скользящего $S=1$, период упреждения (шаг или глубина прогноза) $m=1$.) и для многофакторного прогнозирования (приложение 1).
5. Создать и обучить нейронную сеть для решения задачи многофакторного прогнозирования (исходные данные взять из практической работы № 2). Сравнить полученный результат по МГУА и НС.

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.neural_network import MLPRegressor
5 from sklearn.metrics import mean_absolute_error
6 from sklearn.preprocessing import MinMaxScaler

```



```
1 np.random.seed(42)
2
3 series1 = np.random.uniform(0, 9, size=1000)
4 series2 = np.random.uniform(0, 99, size=1000)
5
6 ts = series1.copy()
```

```
1 def create_dataset(ts, window_size=25, horizon=2):
2     X, y = [], []
3     for i in range(len(ts) - window_size - horizon + 1):
4         X.append(ts[i:i + window_size])
5         y.append(ts[i + window_size:i + window_size + horizon])
6     return np.array(X), np.array(y)
7
8 window_size = 25
9 horizon = 2
10 X, y = create_dataset(ts, window_size, horizon)
11
12 X_train = X[:8]
13 y_train = y[:8]
14
15 X_test = X[8:]
16 y_test = y[8:]
```

```
1 mlp1 = MLPRegressor(
2     hidden_layer_sizes=(10,),
3     activation='relu',
4     solver='adam',
5     max_iter=1000,
6     random_state=42
7 )
8
9 mlp1.fit(X_train, y_train)
```

▼ **MLPRegressor** ⓘ

MLPRegressor(hidden_layer_sizes=(10,), max_iter=1000, random_state=42)

```
1 y_pred = mlp1.predict(X_test)
2
3 # MAE по всем прогнозируемым шагам (усреднённый по горизонту)
4 mae = mean_absolute_error(y_test, y_pred)
5 print(f"MAE = {mae:.4f}")
6
7 # Показать первые прогнозы
8 print("\nПример прогноза (первые 3 образца):")
9 for i in range(3):
10     print(f"Истинные: {y_test[i]}, Прогноз: {y_pred[i]}")
```

MAE = 2.9309

Пример прогноза (первые 3 образца):

Истинные: [8.53996984 8.6906883], Прогноз: [4.39765038 2.483745]

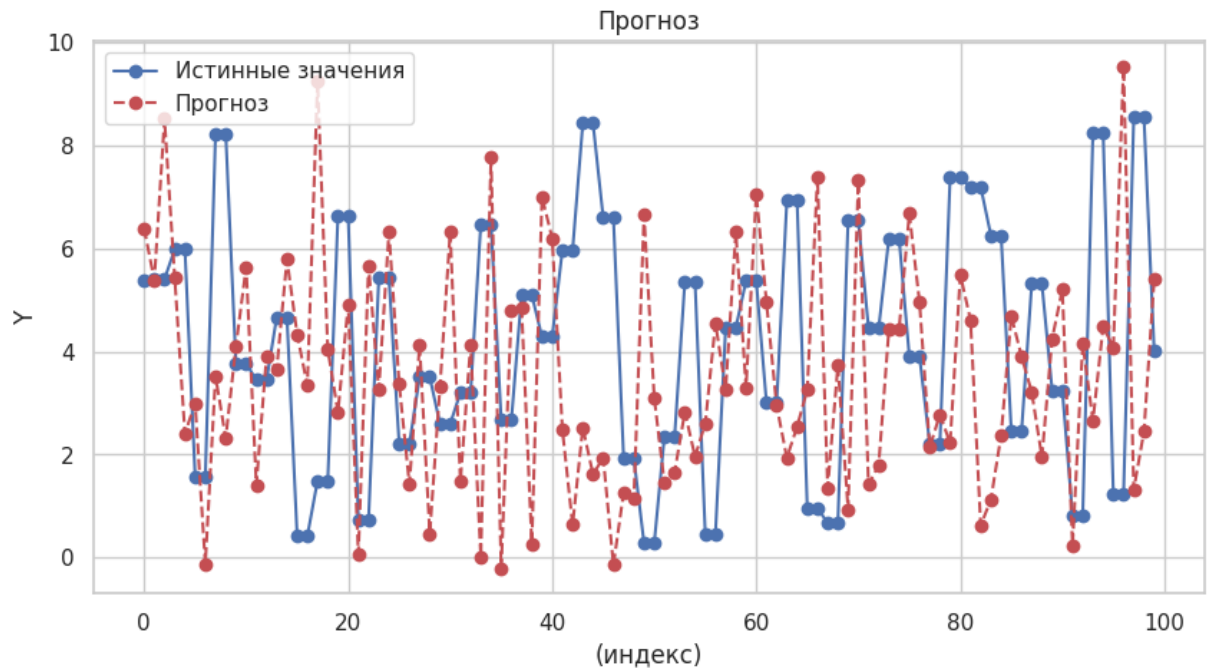
Истинные: [8.6906883 7.27557613], Прогноз: [1.07644086 2.93799317]

Истинные: [7.27557613 2.74152392], Прогноз: [2.9186153 1.34632798]

```

1 plt.figure(figsize=(10, 5))
2 plt.plot(y_test[-50:].flatten(), label='Истинные значения', mar
3 plt.plot(y_pred[-50:].flatten(), 'ro--', label='Прогноз')
4 plt.title('Прогноз')
5 plt.xlabel('(индекс)')
6 plt.ylabel('Y')
7 plt.legend()
8 plt.grid(True)
9 plt.show()

```



```

1 ts = series2.copy()
2
3 window_size = 25
4 horizon = 2
5 X, y = create_dataset(ts, window_size, horizon)
6
7 X_train = X[:8]
8 y_train = y[:8]
9
10 X_test = X[8:]
11 y_test = y[8:]
12
13 mlp1 = MLPRegressor(
14     hidden_layer_sizes=(10,),

```

```

15     activation='relu',
16     solver='adam',
17     max_iter=1000,
18     random_state=42
19 )
20
21 mlp1.fit(X_train, y_train)
22
23 y_pred = mlp1.predict(X_test)
24
25 mae = mean_absolute_error(y_test, y_pred)
26 print(f"MAE = {mae:.4f}")
27
28 print("\nПример прогноза (первые 3 образца):")
29 for i in range(3):
30     print(f"Истинные: {y_test[i]}, Прогноз: {y_pred[i]}")
31
32 plt.figure(figsize=(10, 5))
33 plt.plot(y_test[-50:].flatten(), label='Истинные значения', mar
34 plt.plot(y_pred[-50:].flatten(), 'ro--', label='Прогноз')
35 plt.title('Прогноз')
36 plt.xlabel('(индекс)')
37 plt.ylabel('Y')
38 plt.legend()
39 plt.grid(True)
40 plt.show()

```

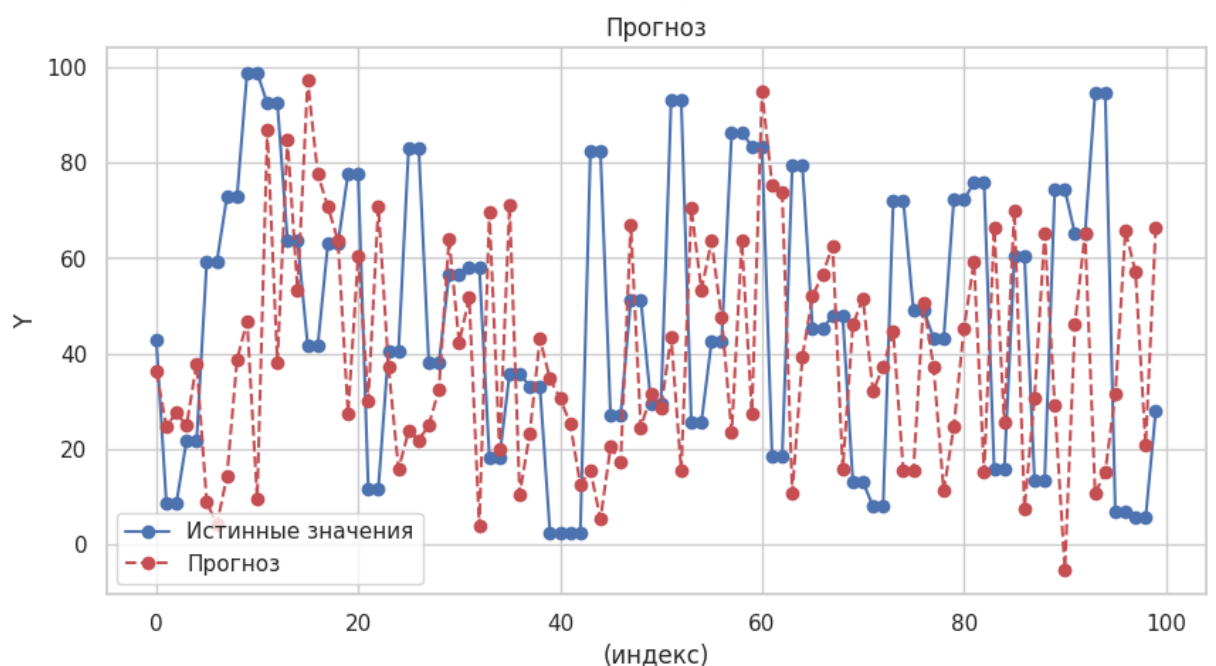
MAE = 31.5503

Пример прогноза (первые 3 образца):

Истинные: [11.93612169 7.61836696], Прогноз: [11.00289926 22.659726

Истинные: [7.61836696 68.93258881], Прогноз: [27.23696095 46.989551

Истинные: [68.93258881 33.64762141], Прогноз: [-1.4968235 36.328351



```
1 data_str = """1990 475.3 0.986 0.978 0.970 1.060 0.880
2 1991 413.5 0.876 0.858 0.870 1.082 0.840
3 1992 401.7 0.699 0.690 0.764 1.104 0.480
4 1993 400.9 0.605 0.619 0.685 1.126 0.475
5 1994 401.3 0.514 0.559 0.566 1.148 0.525
6 1995 402.5 0.483 0.492 0.534 1.170 0.575
7 1996 401.1 0.459 0.501 0.570 1.230 0.582
8 1997 404.4 0.464 0.506 0.593 1.304 0.539
9 1998 406.2 0.478 0.430 0.640 1.336 0.512
10 1999 412.2 0.507 0.587 0.695 1.370 0.519
11 2000 416.5 0.671 0.777 0.730 1.400 0.617
12 2001 425.8 0.801 1.109 0.758 1.430 0.624
13 2002 435.4 0.981 1.267 0.794 1.528 0.634
14 2003 445.4 1.117 1.425 0.830 1.626 0.656
15 2004 454.6 1.254 1.583 0.866 1.724 0.682
16 2005 466.1 1.411 1.741 0.904 1.824 0.729
17 2006 475.2 1.568 1.899 1.075 1.887 0.780"""
18
19 # Преобразуем в DataFrame
20 lines = [line.split() for line in data_str.split('\n')]
21 df = pd.DataFrame(lines, columns=['Year', 'Y', 'X1', 'X2', 'X3', 'X4', 'X5'])
22 df = df.astype(float)
23 df = df.reset_index(drop=True)
24
25 # Целевая переменная – первый столбец после года (Y)
26 Y = df['Y'].values
27 X_multifactor = df[['X1', 'X2', 'X3', 'X4', 'X5']].values
```

```

1 def create_univariate_dataset(ts, window=4, horizon=1):
2     X, y = [], []
3     for i in range(len(ts) - window - horizon + 1):
4         X.append(ts[i:i + window])
5         y.append(ts[i + window])
6     return np.array(X), np.array(y)
7
8 X_uni, y_uni = create_univariate_dataset(Y, window=4, horizon=1)
9
10 split = len(X_uni) - 3
11 X_uni_train, X_uni_test = X_uni[:split], X_uni[split:]
12 y_uni_train, y_uni_test = y_uni[:split], y_uni[split:]
13
14 # Обучение
15 mlp_uni = MLPRegressor(hidden_layer_sizes=(8,), max_iter=1000,
16 mlp_uni.fit(X_uni_train, y_uni_train)
17
18 # Прогноз
19 y_uni_pred = mlp_uni.predict(X_uni_test)
20 mae_uni = mean_absolute_error(y_uni_test, y_uni_pred)
21 print(f"однофакторный прогноз: MAE = {mae_uni:.4f}")

```

однофакторный прогноз: MAE = 16.3174

```

1 split_mf = len(X_multifactor) - 3
2 X_mf_train, X_mf_test = X_multifactor[:split_mf], X_multifactor[split_mf:]
3 y_mf_train, y_mf_test = Y[:split_mf], Y[split_mf:]
4
5 mlp_mf = MLPRegressor(hidden_layer_sizes=(10,), max_iter=2000,
6 mlp_mf.fit(X_mf_train, y_mf_train)
7
8 # Прогноз HC
9 y_mf_pred = mlp_mf.predict(X_mf_test)
10 mae_mf = mean_absolute_error(y_mf_test, y_mf_pred)
11
12 # МГУА
13 from sklearn.linear_model import LinearRegression
14 lr = LinearRegression()
15 lr.fit(X_mf_train, y_mf_train)
16 y_mgu_pred = lr.predict(X_mf_test)
17 mae_mgu = mean_absolute_error(y_mf_test, y_mgu_pred)
18
19 print(f"нейросеть (многофакторная): MAE = {mae_mf:.4f}")
20 print(f"МГУА : MAE = {mae_mgu:.4f}")
21
22 # Вывод сравнения
23 if mae_mf < mae_mgu:
24     print("Нейросеть точнее!")
25 else:
26     print("МГУА точнее")

```

```

нейросеть (многофакторная): MAE = 278.2807
МГУА : MAE = 3.9065
МГУА точнее

```

```

1 plt.figure(figsize=(10, 5))
2 plt.plot(Y, label='Истинные значения', marker='o')
3 plt.plot(range(len(Y)-3, len(Y)), y_mf_pred, 'ro--', label='Про
4 plt.plot(range(len(Y)-3, len(Y)), y_mgu_pred, 'x-', label='Прог
5 plt.title('Прогноз объёма (Y) на последние 3 года')
6 plt.xlabel('Год (индекс)')
7 plt.ylabel('Y')
8 plt.legend()
9 plt.grid(True)
10 plt.show()

```

